

数字逻辑 与 部件设计

Digital Logic & Component Design

1. 简介、进制+码制

xgsun@fudan.edu.cn

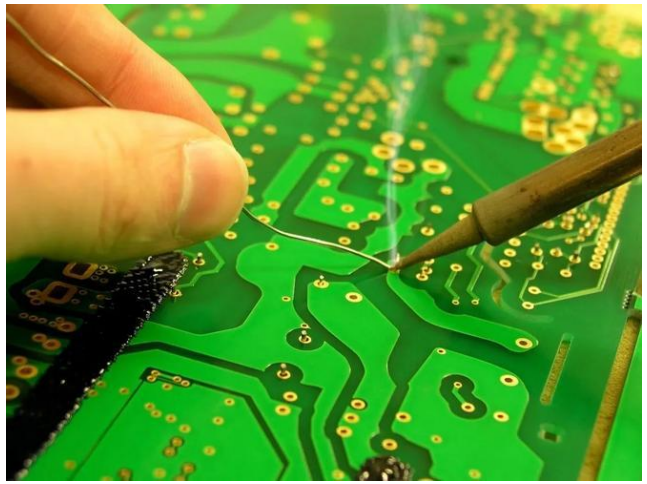
孙晓光

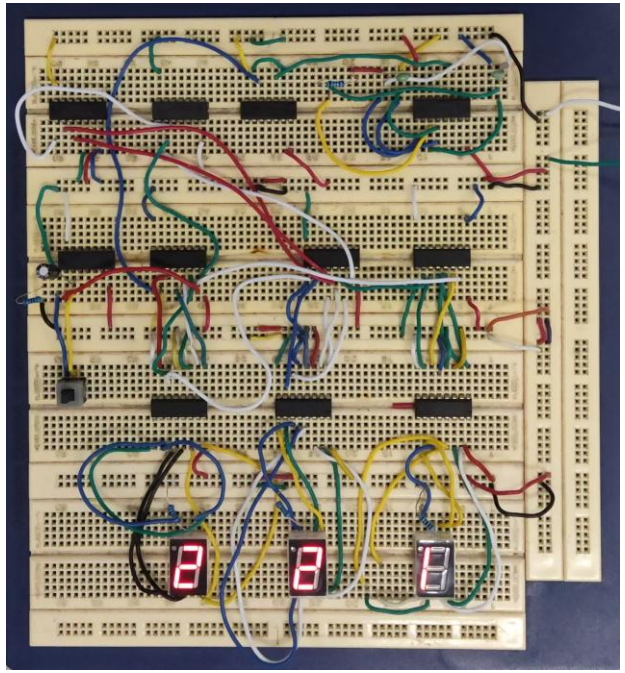
2026-9-7

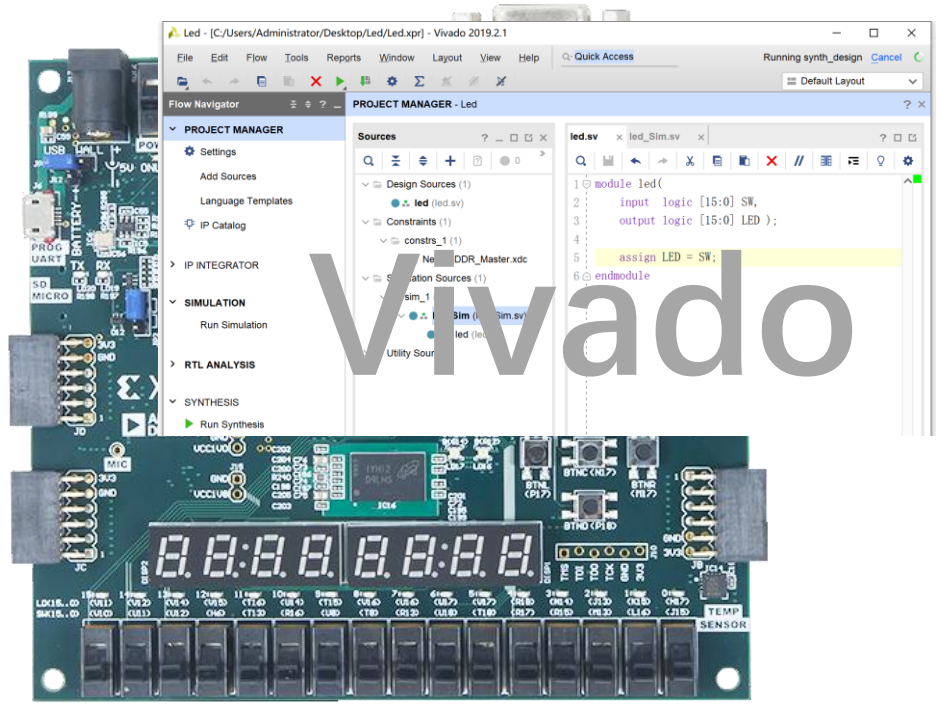
1

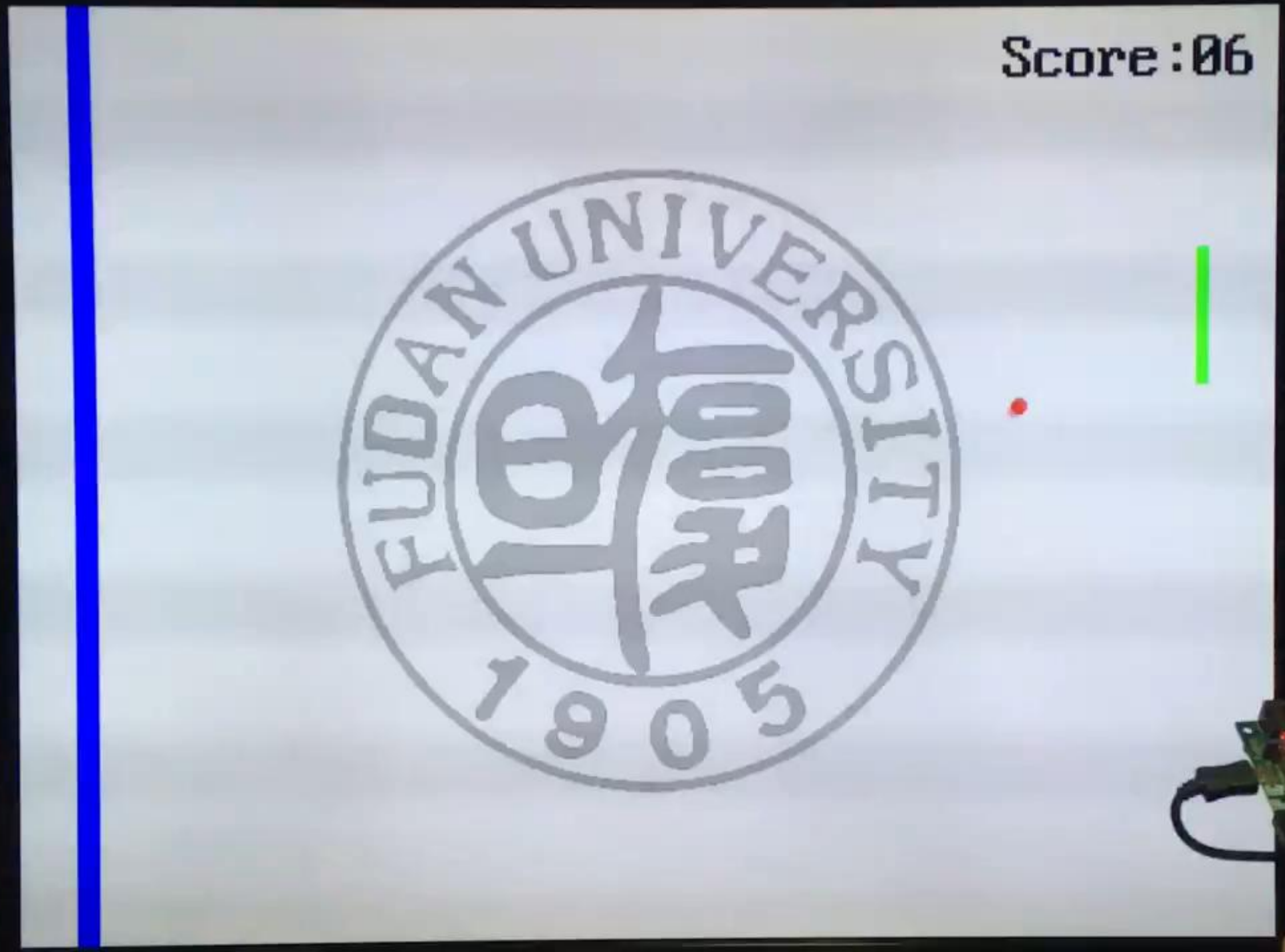
课程简介

硬件 电路 设计






Led - [C:/Users/Administrator/Desktop/Led/Led.xpr] - Vivado 2019.2.1
File Edit Flow Tools Reports Window Layout View Help
Quick Access
Running synth_design Cancel
Default Layout
Flow Navigator
PROJECT MANAGER - Led
PROJECT MANAGER
Settings
Add Sources
Language Templates
IP Catalog
IP INTEGRATOR
SIMULATION
Run Simulation
RTL ANALYSIS
SYNTHESIS
Run Synthesis
Sources
Design Sources (1)
led (led.sv)
Constraints (1)
constrs_1 (1)
led.sv
led_Sim.sv
module led(
input logic [15:0] SW,
output logic [15:0] LED);
assign LED = SW;
endmodule
Vivado


Score:06
FUDAN UNIVERSITY
1905

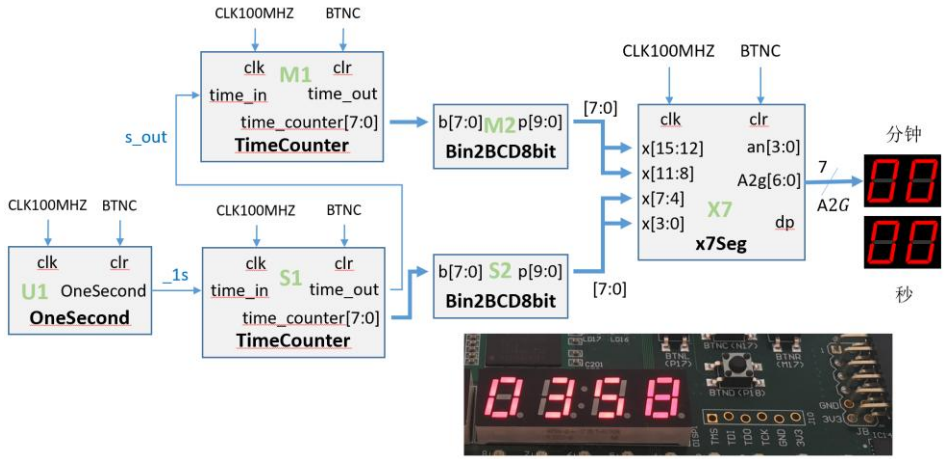
组合逻辑：加法器、选择器、比较器、译码器、编码器、校验器 ……

时序逻辑：锁存器、触发器、寄存器、移位器、计数器 ……





应用



部件实验：ALU、存储器、总线控制器

系统实验：**交通灯控制器、电梯控制器、密码锁、电子秒表、乒乓球游戏、CPU**

课程 **特点**

简单易学

重 要

实践性强



计算机系统结构实验

计算机系统结构

计算机组成原理实验

计算机组成原理

数字逻辑实验

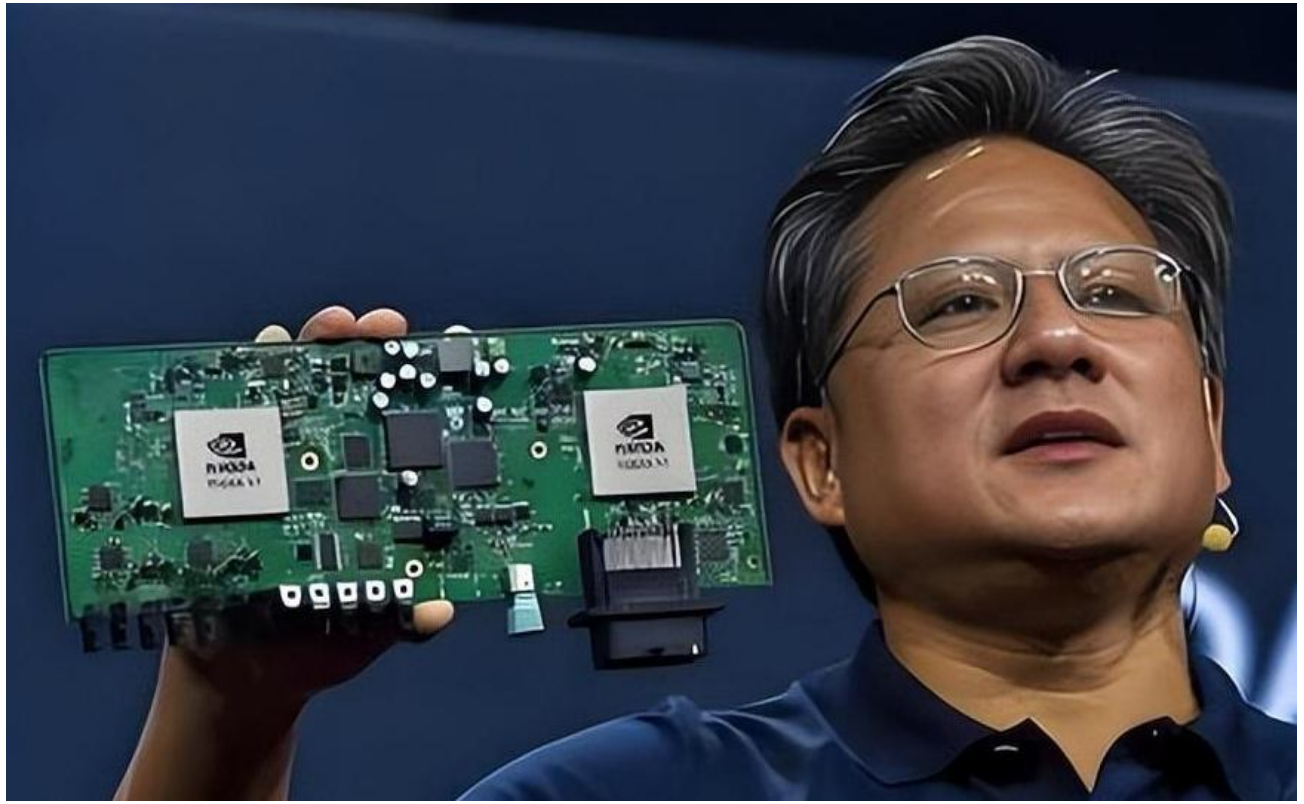
数字逻辑

微机原理+接口 (x86)

嵌 入 式 系 统 (ARM)

AI 硬件的基石

- GPU



- FPGA 上实现 YOLO、大语言模型

学习方法

① 重点：**基本原理、方法**

各类数字集成电路，重点掌握使用方法（输入、输出）

② 用工程近似方法处理工程实际问题

抓住主要因素，忽略次要因素，保证在误差允许范围内

③ 培养实验调试能力

实验有助于加深对理论的理解；大规模设计必须使用EDA工具

教学目的

- 以**逻辑代数**为工具，
- 对**逻辑电路**进行**分析**和**设计**，
- 掌握**数字系统**的**设计**过程，
- 为**计算机**应用奠定扎实的学科基础。

基本要求

① 掌握数字电路**分析**的方法

② 掌握数字电路**设计**的基本理论

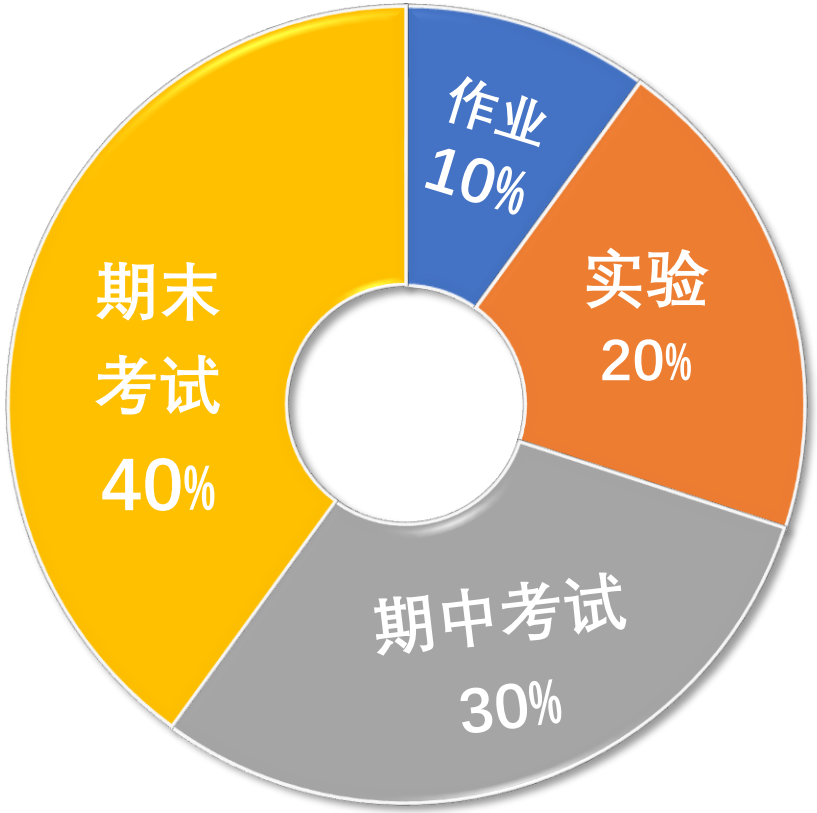
③ 掌握Verilog语言、Vivado设计软件

④ 了解实现数字电路的工艺器件原理

⑤ 了解数字系统设计的发展现状

⑥ 培养对硬件设计的兴趣

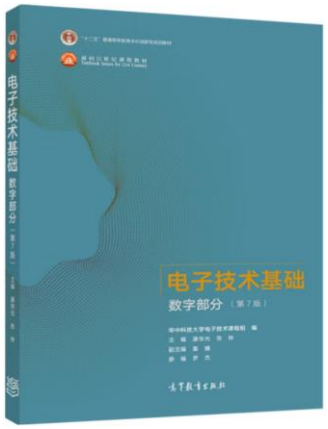
考核方式: 闭卷笔试



每周一交作业

课件下载: http://elearning.fudan.edu.cn (新版)

参考书目



康华光 华中科大 7th, 2021









[M. Morris Mano](#) 6th, 2022

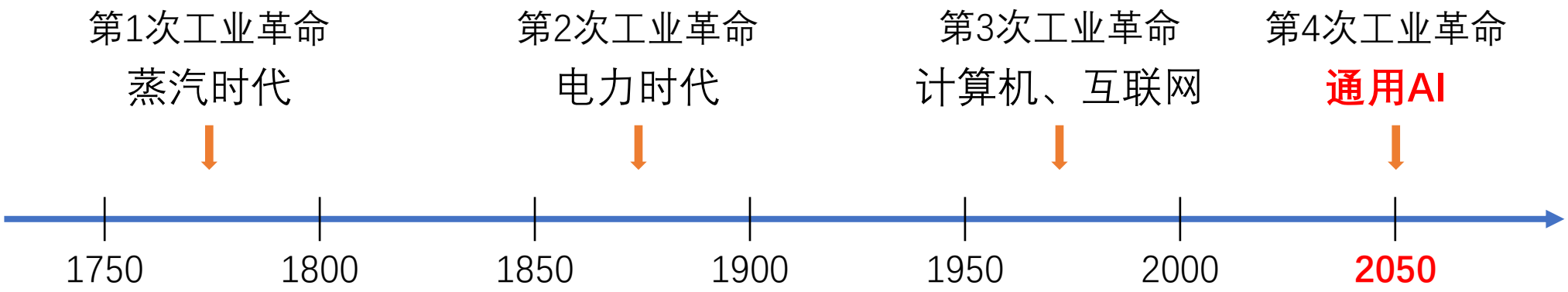






数字系统 Digital System

- **数字系统**：一个传输、存储、处理**数字信号**的系统，由逻辑电路构成。
- **人类已经进入到 数 字 时 代 !**



数字电路 Digital Circuits

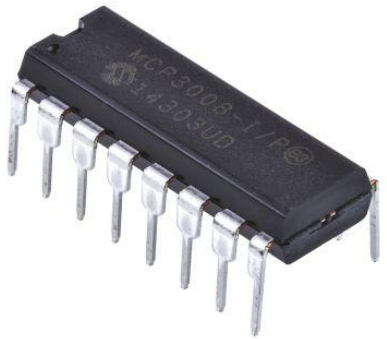
数字电路：用来处理数字信号的电子线路。 又称：**数字逻辑电路、逻辑电路**

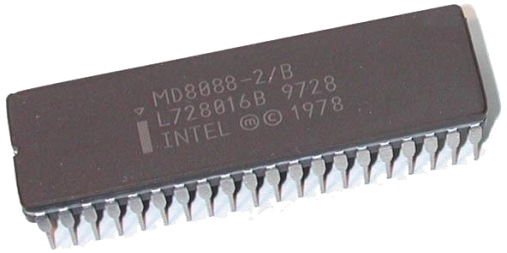
特点

- 工作信号是二值信号，器件工作在开、关状态
- 电路结构简单、功耗低，便于集成制造，价格低
- 工作速度快、精度高、功能强、可靠性好





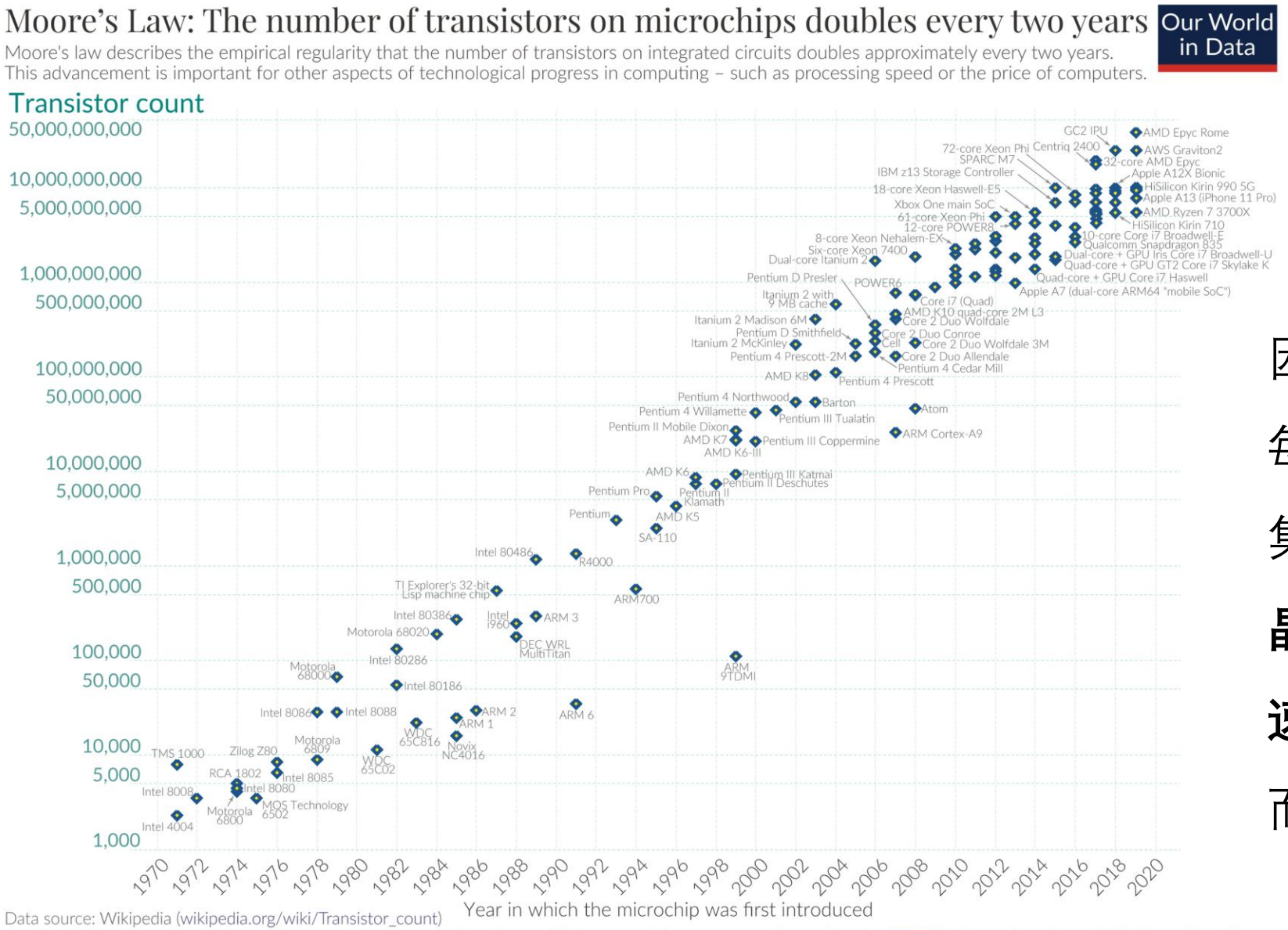






1907 电子真空管 → 1947 晶体管 → 1958 SSI 小规模**集成电路** → MSI 中规模**集成电路** → 1969-1975 VLSI (超)大中规模**集成电路**

摩尔定律



因集成电路技术的不断改进，

每18-24个月（2年），

集成电路芯片上的

晶体管数将翻一番，

速度将提高一倍，

而**价格**将降低一半。

(模拟量) 连续量 和 离散量 (数字量)

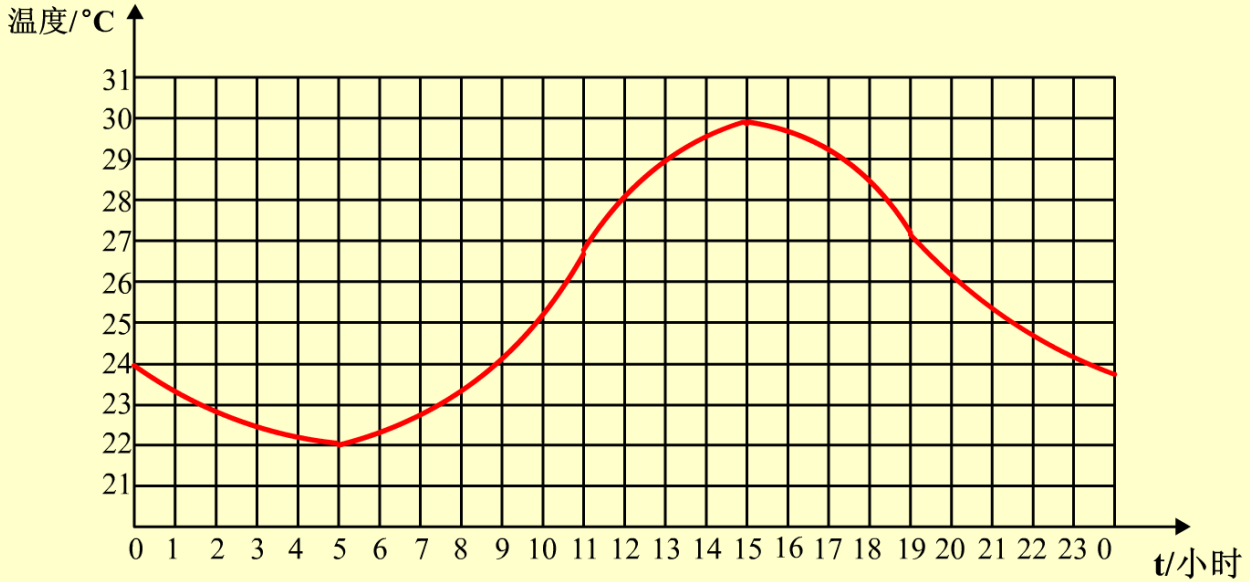


图1.1 温度变化的连续量曲线图

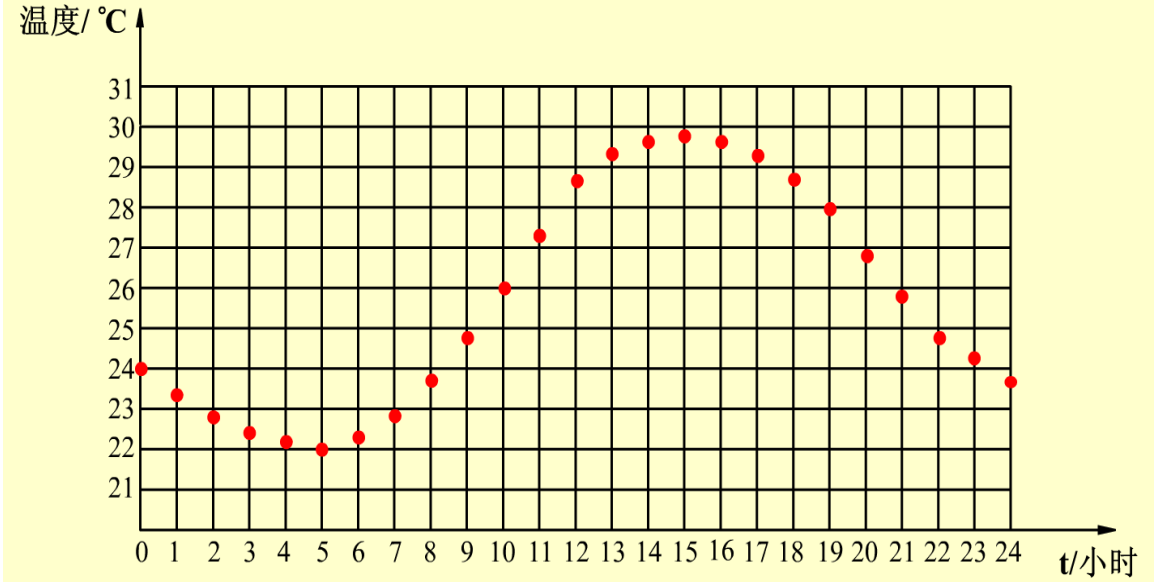


图1.2 温度变化的离散量曲线图



数字量具有精度高、高效**传输**、易**存储**、易**处理**等优点。

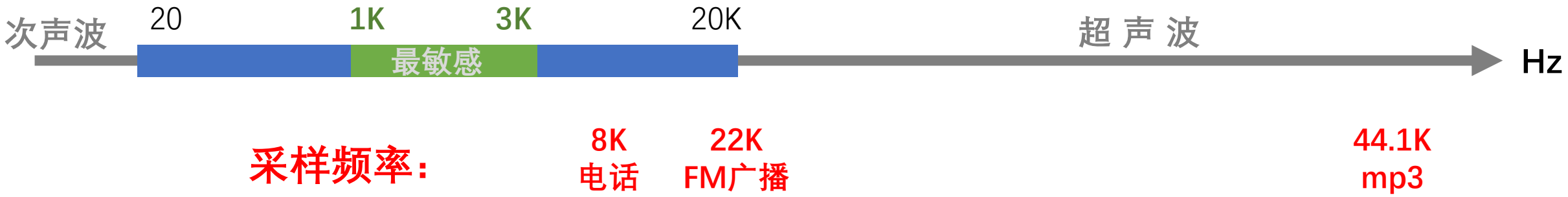
数字是信息的最佳表达形式



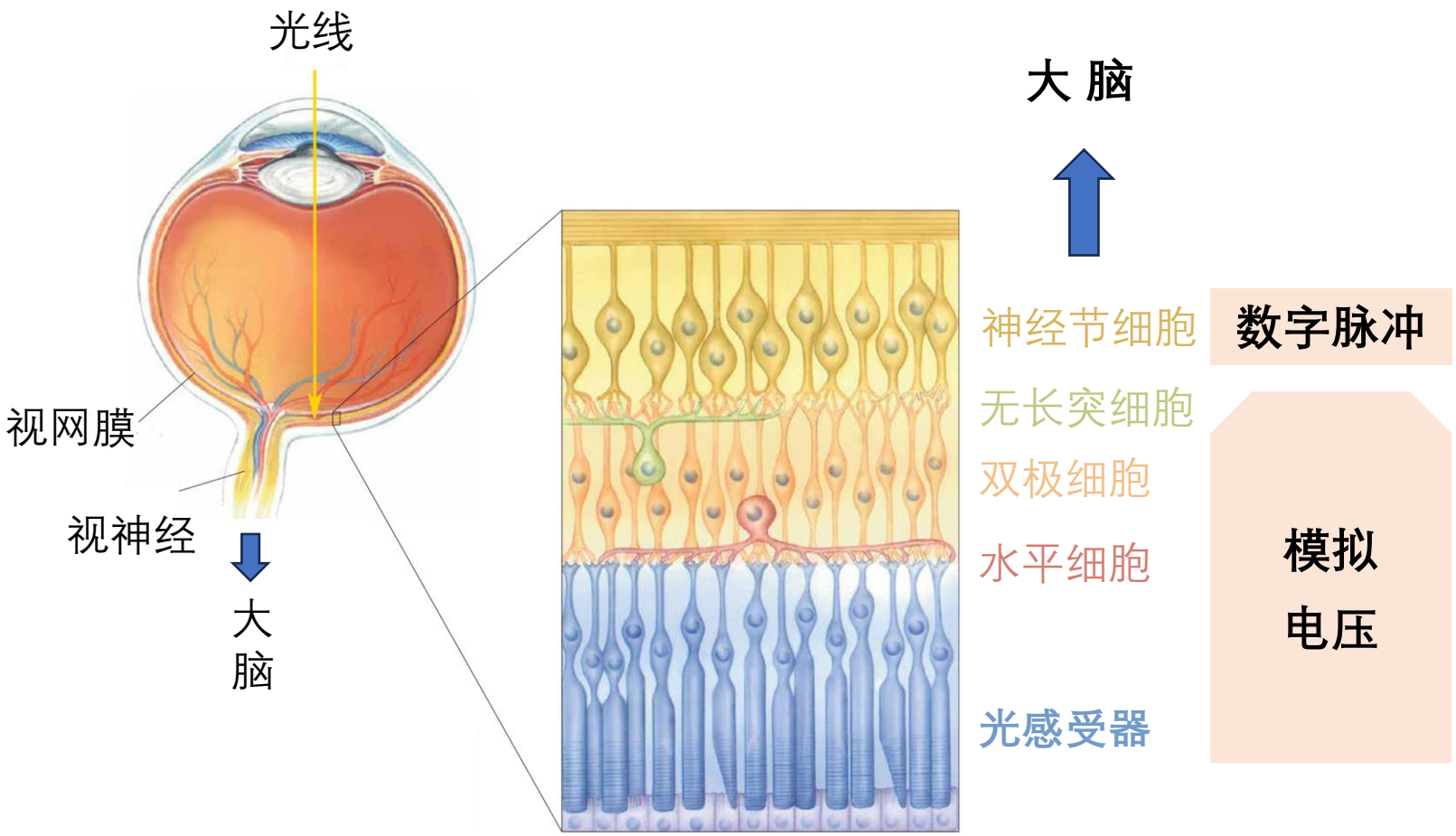
采样定理

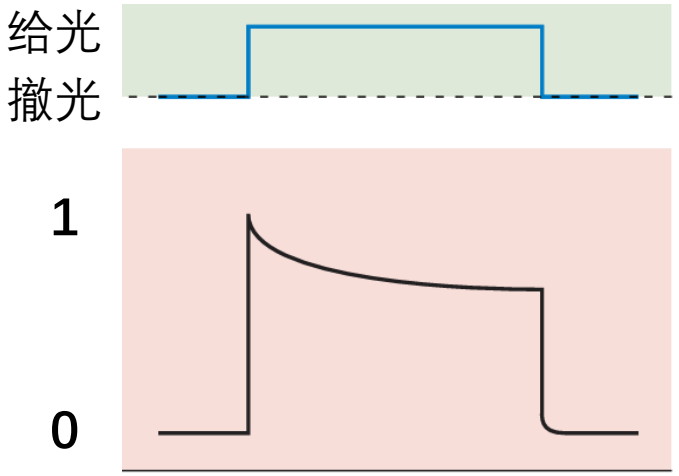
在进行模拟/数字信号的转换过程中，当**采样频率**大于**信号中最高频率**的**2**倍时，采样之后的数字信号完整地保留了原始信号中的信息。 一般采用2.56～4倍。





大脑也是**数字**信号





- **抗衰减**：视神经太长
- **抗噪声**：生物内部太“吵”

优点

- ✓ **并行计算**
- ✓ **信息编码灵活**
- ✓ **传输时能量效率高**

2

进制

十进制编码特点

状态多(0~9，共10个)、**运算组合状态**多

如，两个数加法组合状态数 = $C_{10}^{2} + 10 = 55$

0+0	0+1	0+2	0+3	0+4	0+5	0+6	0+7	0+8	0+9
	1+1	1+2	1+3	1+4	1+5	1+6	1+7	1+8	1+9
		2+2	2+3	2+4	2+5	2+6	2+7	2+8	2+9
			3+3	3+4	3+5	3+6	3+7	3+8	3+9
				4+4	4+5	4+6	4+7	4+8	4+9
					5+5	5+6	5+7	5+8	5+9
						6+6	6+7	6+8	6+9
							7+7	7+8	7+9
								8+8	8+9
									9+9

二进制 Binary

优点：运算简单、物理实现容易、存储、传输方便可靠

缺点：位数太长，书写、记忆和阅读不方便

$$10110_2 = 1\times 2^4 + 0\times 2^3 + 1\times 2^2 + 1\times 2^1 + 0\times 2^0 = 22_{10}$$

one sixteen　no eight　one four　one two　no one

按权展开法：$(N)_r = \left(a_{n-1}\times r^{n-1} + a_{n-2}\times r^{n-2} + \cdots + a_0\times r^0 + \cdots + a_{-m}\times r^{-m}\right)_{10}$

r: 基数

位置记数：$(N)_r = (a_{n-1}\ a_{n-2}\cdots a_1\ a_0\ a_{-1}\cdots a_{-m})_r = \sum_{i=-m}^{n-1} a_i \times r^i$

节省设备

如，篮球计分系统，需要记3位数。

- 采用十进制，需要 30 个记分牌。
- 采用二进制，只需 20 个记分牌。

$2^{10} = 1024 >$ 999



r: 基数，n:位数

$\boldsymbol{n \times r}$: 所需设备量

$\boldsymbol{N}$: 最大信息量 ($N = \boldsymbol{r^n}$)

$\ln \boldsymbol{N} = \boldsymbol{n} \cdot \ln \boldsymbol{r}$　　令：$C = \ln N$

$C = n \cdot \ln r$　　两边同乘r

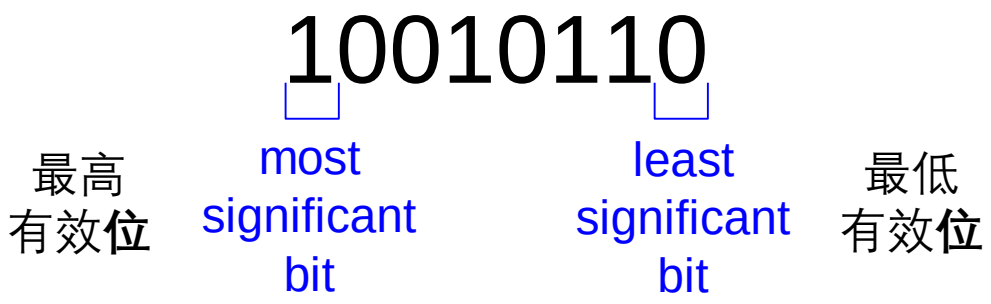
$rC = \boldsymbol{n} \times \boldsymbol{r} \cdot \ln r$

$\boldsymbol{n \times r} = \dfrac{rC}{\ln r}$　　$\left(\dfrac{rC}{\ln r}\right)' = 0$

$\ln r = 1$　　$\boldsymbol{r = e = 2.718}$

bit, Byte, Nibble, Word

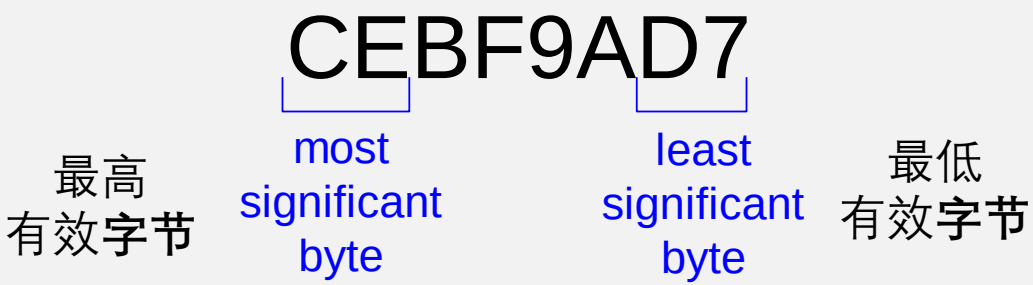
- **bit 位**



- **Byte**、Nibble
  字节　半字节

  8位一组





- Word
  字

大小取决于微处理器的结构，如，32位微处理器使用32位字。

0 扩展
Zero-Extension

- Zeros copied to msb's 最高有效位
- Value changes for negative numbers

- **Example 1:**
  - 4-bit value = $0011_2 = 3_{10}$
  - 8-bit zero-extended value: $00000011 = 3_{10}$

- **Example 2:**
  - 4-bit value = $(1011)_{补} = -5_{10}$
  - 8-bit zero-extended value: $(00001011)_{补} = 11_{10}$

符号扩展
Sign-Extension

- Sign bit copied to msb's
- Number value is same

- **Example 1:**
  - 4-bit representation of 3 = **0**011
  - 8-bit sign-extended value: 0000**0**011

- **Example 2:**
  - 4-bit representation of $-5_{10} = (\mathbf{1}011)_{补}$
  - 8-bit sign-extended value: $(1111\mathbf{1}011)_{补}$

（无符号）二进制的四则运算

加法规则 $0+0=0,\ 0+1=1,\ 1+0=1,\ 1+1=0$（进位为1）

减法规则 $0-0=0,\ 1-1=0,\ 1-0=1,\ 0-1=1$（借位为1）

乘法规则 $0\times0=0,\ 0\times1=0,\ 1\times0=0,\ 1\times1=1$

除法规则 $0\div1=0,\ 1\div1=1$

```
  11001
+   101
-------
  11110
```

```
  11001
-   101
-------
  10100
```

```
    11001
×     101
---------
    11001
   00000
+ 11001
---------
  1111101
```

```
        101
   ------
101 ) 11001
    - 101
    -----
      101
    - 101
    -----
        0
```

将减法转换成加法

时钟



$9 - 7 = 2$

$7 + 5 = 12$

$9 + 5 = 14$

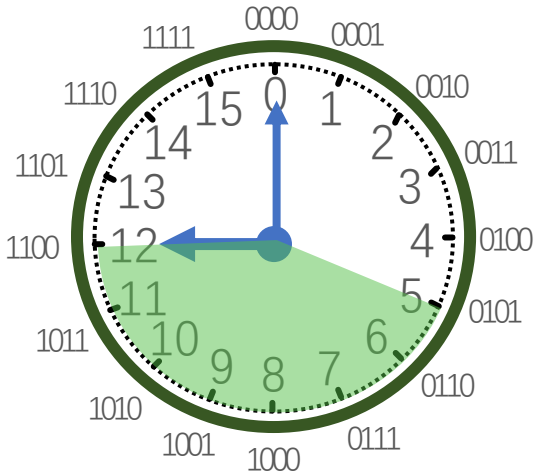
7是5 对于模12的补码



(模：系统的计数范围)

在舍弃进位的情况下，减去某数可以用加上它的补码代替！

4位二进制



$1100 - 0111 = 0101$

$12 - 7 = 5$

$7 + 9 = 16$

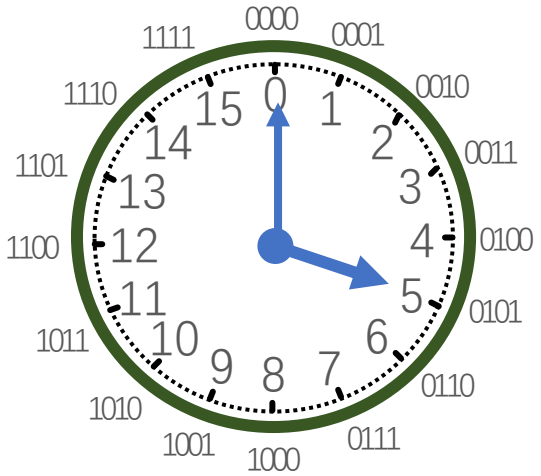
0111

1001

10000

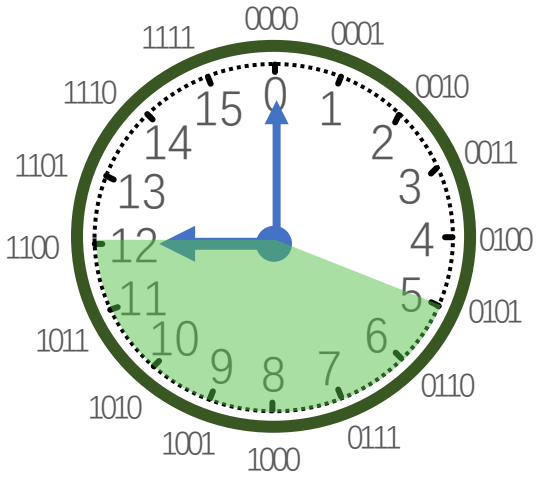
$12 + 9 = 21$

$1100 + 1001 = 1\,0101$



如何求补码？　补码　互补

4位二进制



$1100 - 0111 = 0101$

$12 - 7 = 5$

7+9=16

0111

1001

10000

$12 + 9 = 21$

$1100 + 1001 = 10101$

求补码时还有减法？！

	10000	16
−	0111	7
	1001	9

n位**二进制**数N,

(2的补码) $(N)_{补} = 2^n - N$

(1的补码) $(N)_{反} = (2^n - 1) - N$

$(N)_{补} = (N)_{反} + 1$

反码：

+1

‖

补码：

	1111	$(16-1)$
−	0111	7
	1000	$(9-1)$ 反码
+	1	
	1001	9

将减法转换成加法的代价

时钟



$$9 - 7 = 2$$

$$7 + 5 = 12$$

$$9 + 5 = 14$$



7是5 对于模12的补码。

① 求减数(7)的反码

② 求减数的补码（=反码+1）

③ 被减数+减数补码

④ 去掉最高位进位

有了**补码**的概念，

可以将**减法**转换为**加法**，

这样比**笔算**减法简单。

（逢十进一） 十进制、二进制 （逢二进一）

【例1】将**十进制数**写成**权表示**的形式：

$$(26.75)_{10} = 2 \times 10^{1} + 6 \times 10^{0} + 7 \times 10^{-1} + 5 \times 10^{-2}$$

$$(S)_{10} = k_n \times 10^{n-1} + \cdots + k_1 \times 10^{0} + k_0 \times 10^{-1} + \cdots + k_{-m} \times 10^{-m-1}$$

k_i：0~9　　m, n：正整数，表示权

【例2】将**二进制数**写成**权表示**的形式：

$$(101.011)_{2} = 1 \times 2^{2} + 0 \times 2^{1} + 1 \times 2^{0} + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}$$

$$(S)_{2} = k_n \times 2^{n-1} + \cdots + k_1 \times 2^{0} + k_0 \times 2^{-1} + \cdots + k_{-m} \times 2^{-m-1}$$

k_i：0或1　　m, n：正整数，表示权

（逢八进一）八进制、十六进制（逢十六进一）

【例3】将**八进制数**写成**权表示**的形式：

$$(\mathbf{67.731})_8 = \mathbf{6} \times 8^1 + \mathbf{7} \times 8^0 + \mathbf{7} \times 8^{-1} + \mathbf{3} \times 8^{-2} + \mathbf{1} \times 8^{-3}$$

$$(\boldsymbol{S})_8 = \boldsymbol{k_n} \times 8^{n-1} + \cdots + \boldsymbol{k_1} \times 8^0 + \boldsymbol{k_0} \times 8^{-1} + \cdots + \boldsymbol{k_{-m}} \times 8^{-m-1}$$

$\boldsymbol{k_i}$：0~7　　$\boldsymbol{m, n}$：正整数，表示权

【例4】将**十六进制数**写成**权表示**的形式：

$$(\boldsymbol{8AE6})_{16} = \mathbf{8} \times 16^3 + \boldsymbol{A} \times 16^2 + \boldsymbol{E} \times 16^1 + \mathbf{6} \times 16^0$$

$$(\boldsymbol{S})_{16} = \boldsymbol{k_n} \times 16^{n-1} + \cdots + \boldsymbol{k_1} \times 16^0 + \boldsymbol{k_0} \times 16^{-1} + \cdots + \boldsymbol{k_{-m}} \times 16^{-m-1}$$

$\boldsymbol{k_i}$：0~9, A, B, C, D, E, F　　$\boldsymbol{m, n}$：正整数，表示权

相互转换：**按权相加法**

【二进制、八进制、十六进制】➔【十进制】

【例5】 $(1010)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 8 + 2 = (10)_{10}$

反向操作：可以将**十进制** 转换为 **二进制**，但计算麻烦

$(167)_8 = 1 \times 8^2 + 6 \times 8^1 + 7 \times 8^0 = 64 + 48 + 7 = (119)_{10}$

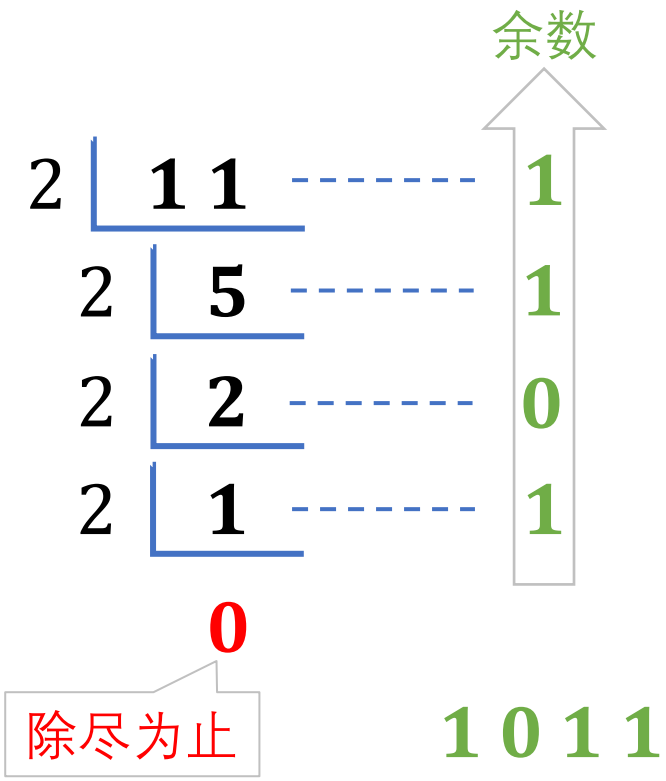
$(0.42)_8 = 4 \times 8^{-1} + 2 \times 8^{-2} = 0.5 + 0.03125 = (0.53125)_{10}$

$(1C4)_{16} = 1 \times 16^2 + 12 \times 16^1 + 4 \times 16^0 = 256 + 192 + 4 = (452)_{10}$

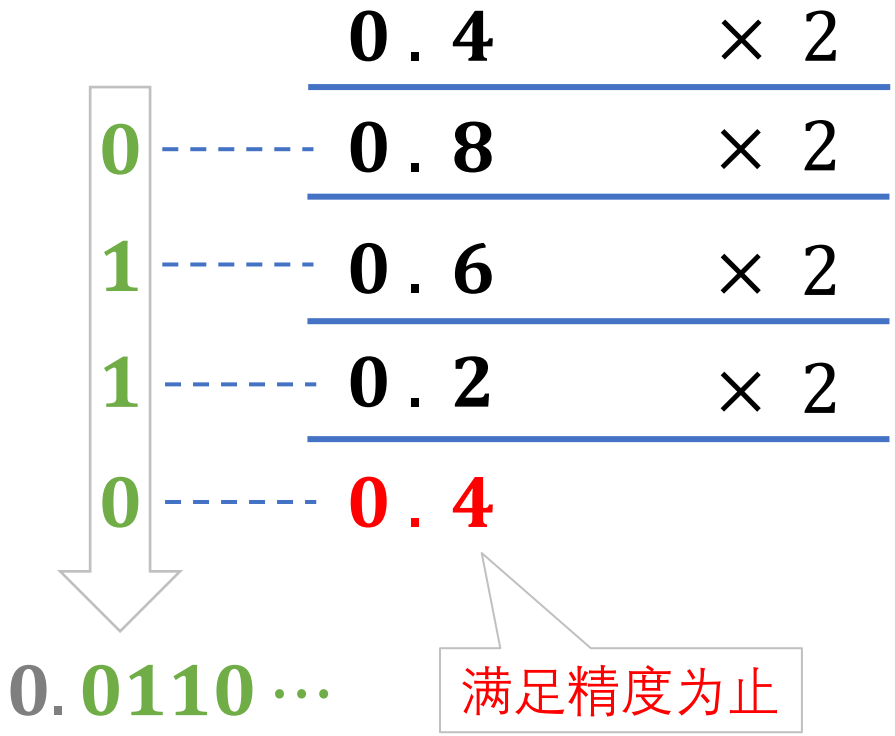
$(0.68)_{16} = 6 \times 16^{-1} + 8 \times 16^{-2} = 0.375 + 0.03125 = (0.40625)_{10}$

基数乘除法：十进制 => 二进制

整数：除2取余



小数：乘2取整

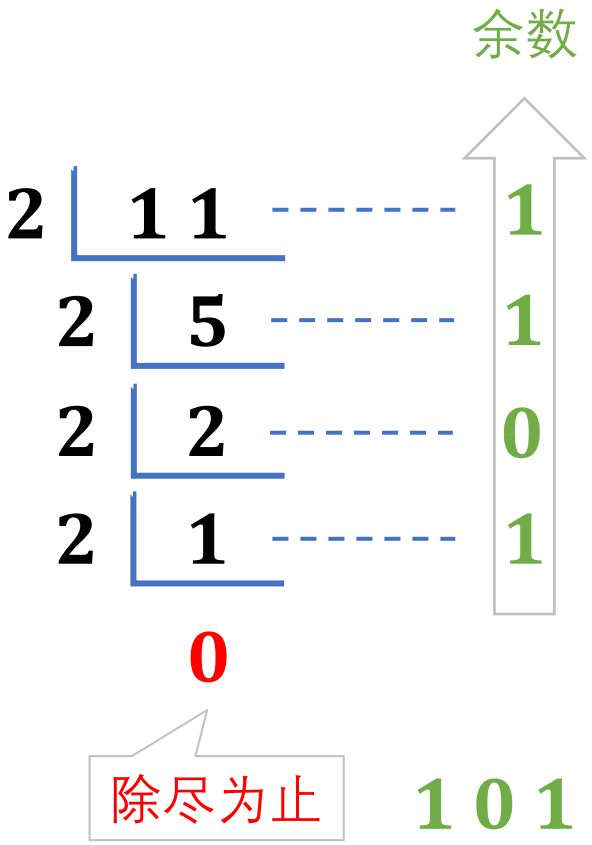


十进制 => 八 进 制：整数除 8取余，小数乘 8取整

十进制 => 十六进制：整数除16取余，小数乘16取整

十进制 => 二进制：整数

整数：除2取余



$$(11)_{10} = (a_{n-1}a_{n-2}\cdots a_1a_0)_2$$

$$= a_{n-1}\times 2^{n-1} + a_{n-2}\times 2^{n-2} + \cdots + a_1\times 2^1 + \boldsymbol{a_0}\times 2^0$$

$$= 2\cdot\left(a_{n-1}\times 2^{n-2} + a_{n-2}\times 2^{n-3} + \cdots + a_1\right) + \boldsymbol{a_0}$$

余数

$$\left(5+\frac{1}{2}\right)_{10} = \left(a_{n-1}\times 2^{n-2} + a_{n-2}\times 2^{n-3} + \cdots + a_1\right) + \frac{\boldsymbol{a_0}}{2}$$

十进制 => 二进制：小数

$$(0.625)_{10} = (0.a_{-1}a_{-2}\cdots a_{-m})_2$$

$$= a_{-1} \times 2^{-1} + a_{-2} \times 2^{-2} + \cdots + a_{-m} \times 2^{-m}$$

$$= \frac{a_{-1}}{2} + \frac{1}{2}(a_{-2} \times 2^{-1} + \cdots + a_{-m} \times 2^{-m+1})$$

$$(1.25)_{10} = a_{-1} + (a_{-2} \times 2^{-1} + \cdots + a_{-m} \times 2^{-m+1})$$

整数

b） 小数乘2取整

```
  0.625
x     2
-------
  1.25
x     2
-------
  0.5
x   2
-------
  1.0
```

整数

二进制、八进制、十六进制

八进制：	2	5	7	.	0	5	5	4
二进制：	010	101	111	.	000	101	101	100

十六进制：	A	F	.	1	6	C
二进制：	1010	1111	.	0001	0110	1100

分小数点，为分割

【例6】 $(67.731)_8 = (110\ 111.111\ 011\ 001)_2$

$(3AB4)_{16} = (0011\ 1010\ 1011\ 0100)_2$

最高位，不足3位补0　　最低位，不足3位补0

【例7】 $(_11\ 111\ 101.010\ 011\ 11_)_2 = (375.236)_8$

最高位，不足4位补0　　最低位，不足4位补0

【例8】 $(_111\ 1101.0100\ 111_)_2 = (7D.4E)_{16}$

带符号二进制数



真值	机器码		
用 ± 表示	符号位：0表示正，1表示负		
	原码	反码	补码
正数 +1010	01010	01010	01010 相同
负数 -1010	11010	10101	10110
	10001010	11110101	11110110
		数值位逐位取反	数值位逐位取反+1
	有+0 和 -0	有+0 和 -0	只有+0
	不便于运算	不便于运算	便于运算

原码的定义

整数 $X = \pm x_{n-1}\, x_{n-2} \cdots x_0$ （x_n 为符号位）

$$[X]_{原} = \begin{cases} X & 0 \le X < 2^n \\ 2^n - X & -2^n < X \le 0 \end{cases}$$

【例】$X_1 = +1101$，$X_2 = -1101$

$[X_1]_{原} = 01101$

$[X_2]_{原} = 2^4 - (-1101)$

$= 10000 + 1101 = 11101$

小数 $X = \pm 0.x_{-1}\, x_{-2} \cdots x_{-m}$

$$[X]_{原} = \begin{cases} X & 0 \le X < 2^0 \\ 2^0 - X & -2^0 < X \le 0 \end{cases}$$

【例】$X_1 = +0.1101$，$X_2 = -0.1101$

$[X_1]_{原} = 0.1101$

$[X_2]_{原} = 2^0 - (-0.1101)$

$= 1.0000 + 0.1101 = 1.1101$

反码的定义

整数 $X = \pm x_{n-1}\, x_{n-2} \cdots x_0$

$$[X]_{反} = \begin{cases} X & 0 \le X < 2^n \\ 2^{n+1} - 1 + X & -2^n < X \le 0 \end{cases}$$

小数 $X = \pm 0.x_{-1}\, x_{-2} \cdots x_{-m}$

$$[X]_{反} = \begin{cases} X & 0 \le X < 2^0 \\ 2 - 2^{-m} + X & -2^0 < X \le 0 \end{cases}$$

【例】$X_1 = +1101$，$X_2 = -1101$

$[X_1]_{反} = 01101$

$[X_2]_{反} = 2^5 - 1 + (-1101)$

$= 11111 - 1101 = 10010$

【例】$X_1 = +0.1101$，$X_2 = -0.1101$

$[X_1]_{反} = 0.1101$

$[X_2]_{反} = 2 - 2^{-4} + (-0.1101)$

$= 10.0000 - 0.0001 - 0.1101$

$= 1.1111 - 0.1101 = 1.0010$

补码的定义

整数 $X = \pm x_{n-1}\, x_{n-2} \cdots x_0$

$$[X]_{补} = \begin{cases} X & 0 \le X < 2^n \\ 2^{n+1} + X & -2^n \le X < 0 \end{cases}$$

【例】$X_1 = +1101$，$X_2 = -1101$

$[X_1]_{补} = 01101$

$[X_2]_{补} = 2^5 + (-1101)$

$= 100000 - 1101 = 10011$

小数 $X = \pm 0.x_{-1}\, x_{-2} \cdots x_{-m}$

$$[X]_{补} = \begin{cases} X & 0 \le X < 2^0 \\ 2 + X & -2^0 \le X < 0 \end{cases}$$

【例】$X_1 = +0.1101$，$X_2 = -0.1101$

$[X_1]_{补} = 0.1101$

$[X_2]_{补} = 2 + (-1101)$

$= 10.0000 - 0.1101 = 1.0011$

补码的运算

$$[x+y]_{补}=[x]_{补}+[y]_{补} \qquad [x-y]_{补}=[x]_{补}+[-y]_{补}$$

【例】$x=+0.1101$，$y=+0.0110$。求 $x-y$

$$[y]_{补}=\mathbf{0}0.0110$$

$$[-y]_{反}=\mathbf{1}1.1001$$

$$[-y]_{补}=\mathbf{1}1.10\mathbf{10}$$

$$+\quad [x]_{补}=\mathbf{0}0.1101$$

$$[x-y]_{补}=\mathbf{1}\mathbf{0}0.0111$$

符号位的进位要**抛弃**

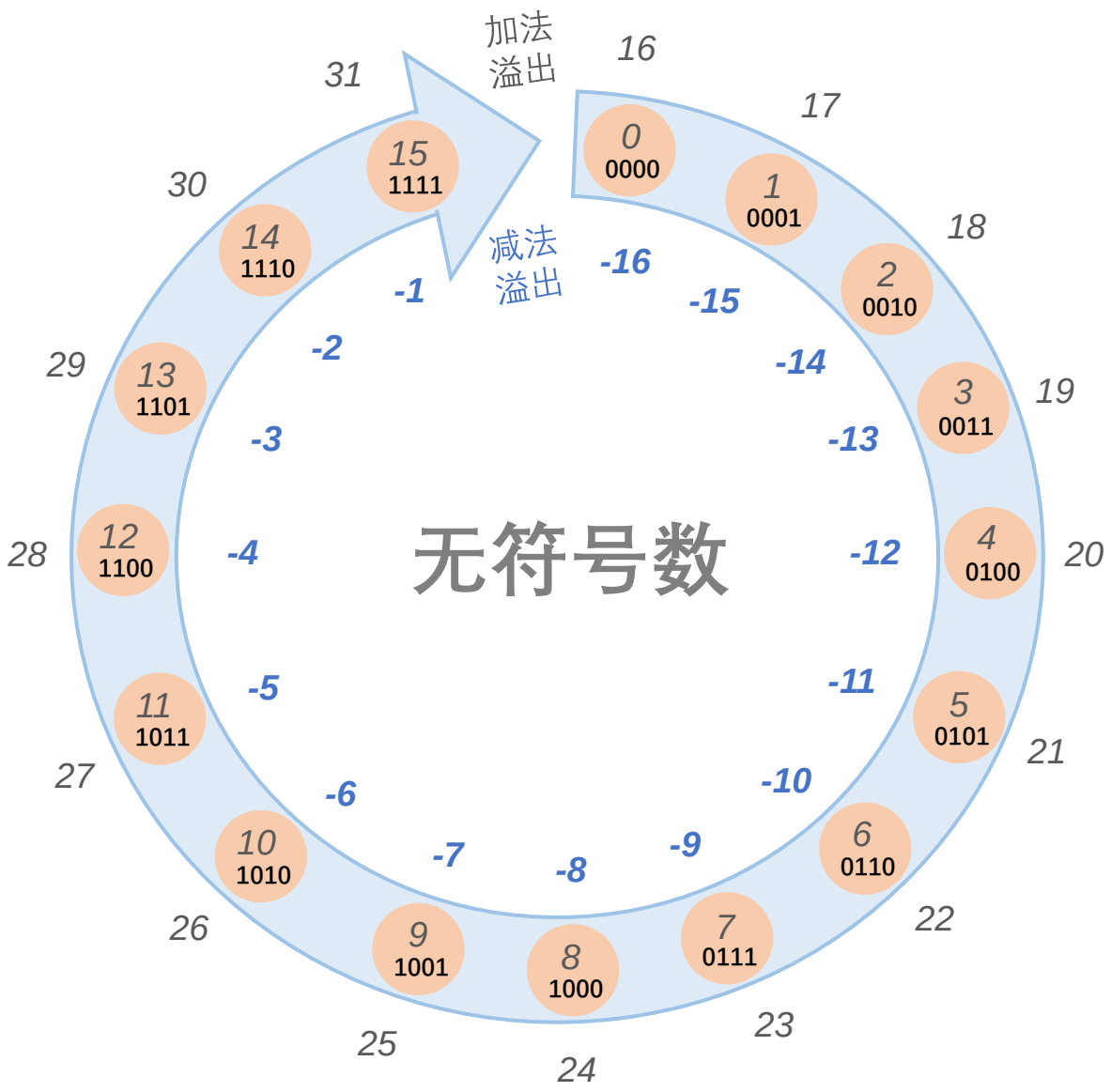
真值：$x-y=+0.0111$

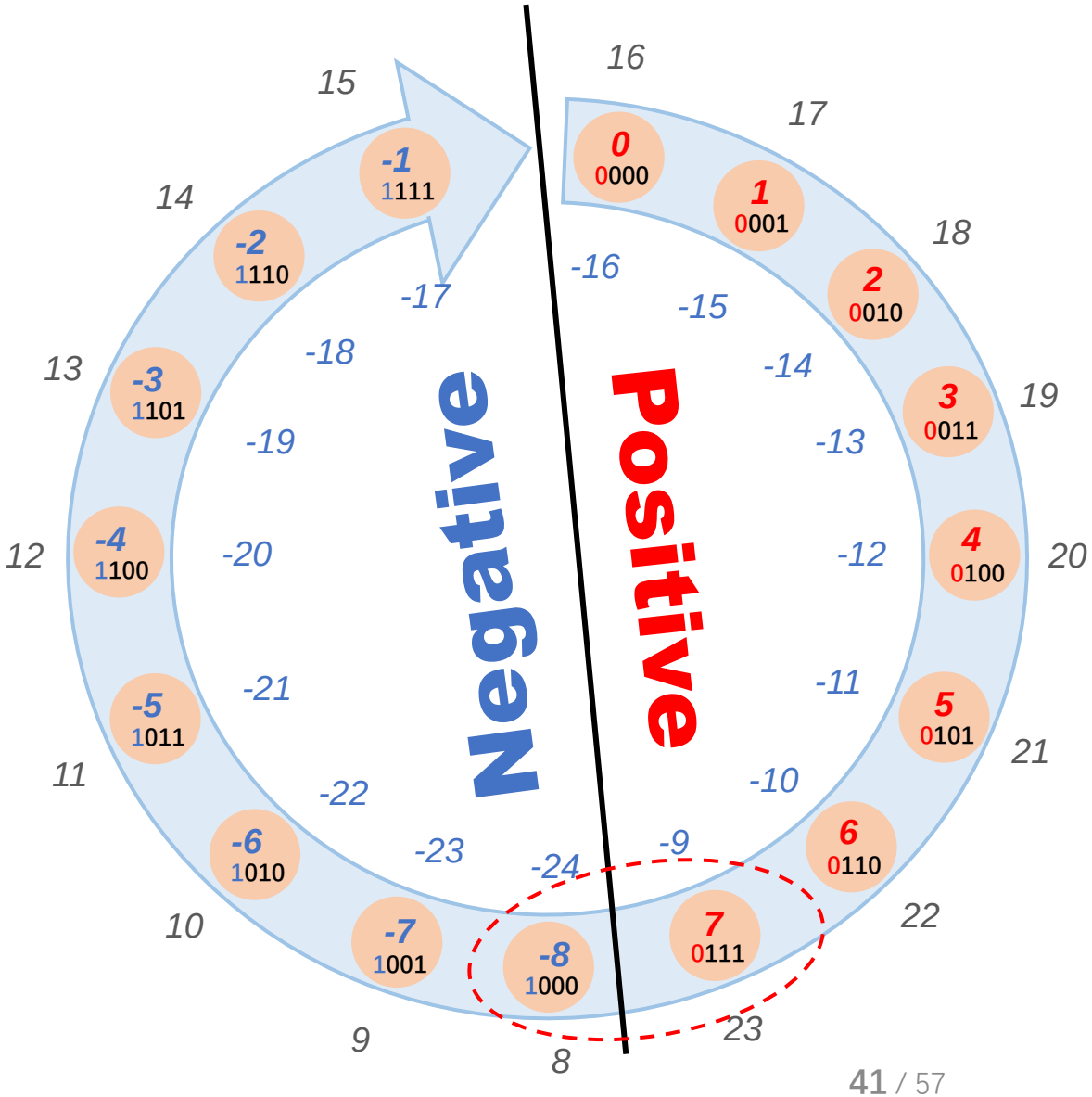
3位**有符号**二进制数 的 原码、反码、补码

十进制	原码	反码	补码
+0	0000	0000	**0000**
+1	0001	0001	**0001**
+2	0010	0010	**0010**
+3	0011	0011	**0011**
+4	0100	0100	**0100**
+5	0101	0101	**0101**
+6	0110	0110	**0110**
+7	0111	0111	**0111**
——	正数的**原码 = 反码 = 补码**		

十进制	原码	反码	补码
-0	1000	1111	——
-1	1001	1110	**1111**
-2	1010	1101	**1110**
-3	1011	1100	**1101**
-4	1100	1011	**1100**
-5	1101	1010	**1011**
-6	1110	1001	**1010**
-7	1111	1000	**1001**
-8	——	——	**1000**

int


加法
溢出
减法
溢出
无符号数
0 0000
1 0001
2 0010
3 0011
4 0100
5 0101
6 0110
7 0111
8 1000
9 1001
10 1010
11 1011
12 1100
13 1101
14 1110
15 1111
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
-1
-2
-3
-4
-5
-6
-7
-8
-9
-10
-11
-12
-13
-14
-15
-16


Negative
Positive
0 0000
1 0001
2 0010
3 0011
4 0100
5 0101
6 0110
7 0111
-8 1000
-7 1001
-6 1010
-5 1011
-4 1100
-3 1101
-2 1110
-1 1111
16
17
18
19
20
21
22
23
8
9
10
11
12
13
14
15
-16
-15
-14
-13
-12
-11
-10
-9
-17
-18
-19
-20
-21
-22
-23
-24

溢出 Overflow

【例】用4位$[-7 \sim 7]_{10}$表示有符号二进制数，计算：

√ 2+3=5

```
  0 0 1 0    2
+ 0 0 1 1    3
-----------
  0 1 0 1    5
```

不能溢出！

× 5+6=11

```
  0 1 0 1    5
+ 0 1 1 0    6
-----------
? 1 0 1 1   11
```

-3

无符号数溢出：最高有效位进位/借位。

有符号数溢出：如果相加两个数的符号相同，但结果的符号与被加数的符号相反。

√ 5+6=11

```
  0 0 0 0  0 1 0 1    5
+ 0 0 0 0  0 1 1 0    6
-----------------
  0 0 0 0  1 0 1 1   11
```

练习1

- 已知：$x = 01010,\ y = 00011,\ z = 00011$

  用补码运算求：$x + y = ?,\ x - z = ?$

$x + y = ?$

$$x_{补} = 01010$$
$$+\quad y_{补} = 00011$$

$$(x + y)_{补} = 01101$$

$$(x + y)_{原} = 01101$$

$$x + y = +1101$$

$x - z = ?$

$$(x)_{补} = 01010$$
$$+\quad (-z)_{补} = 11101$$

$$(x - z)_{补} = 100111$$

$$(x - z)_{原} = 00111$$

$$x - z = +0111$$

验算

$$01010$$
$$-\ 00011$$

$$+0111$$

练习2

- 已知：$x = \mathbf{0}0011,\ y = \mathbf{0}1010$ 用补码运算求：$x - y = ?$

$$x - y = ?$$

$$x_{补} = 00011$$
$$+\ (-y)_{补} = 10110$$
$$[x + (-y)]_{补} = 11001$$
$$[x + (-y)]_{原} = \mathbf{10111}$$
$$x - y = \mathbf{-0111}$$

验　　算

$$x = \mathbf{0}0011$$
$$-\ y = 01010$$
$$\mathbf{11001}$$

$$y = 01010$$
$$-\ x = 00011$$
$$\mathbf{00111}$$

有符号数减法：

- 大数－小数
- 添加负符号

练习3

- **用二进制求：$12-7=5$**

因为12二进制需要4位，加上1位符号位，共需5位。

$$
\begin{array}{rl}
 & (12)_{补}=01100 \\
+ & (-7)_{补}=11001 \\
\hline
 & (12-7)_{补}=\mathbf{00101} \\
 & (12-7)_{原}=\mathbf{00101}
\end{array}
$$

$12-7=+0101=5$

验算

$$
\begin{array}{rl}
 & 12=1100 \\
- & 7=0111 \\
\hline
 & \mathbf{0101}
\end{array}
$$

3

码制

常用的二进制码

码：**数字状态1和0的组合**。用来表示数字、字母、符号等信息，如 ASCII码。

数字系统中的信息：{ **数字信息** / **代码信息** } 数字化**编码**（规定一个具体码组的过程）

- 常用BCD码：8421码、2421码、余3码、84-2-1码……
- ASCII码
- 可靠性编码：格雷码、奇偶校验码

BCD码 Binary Coded Decimal

十进制数	一位热键码	BCD码
0	00000 00001	0000
1	00000 00010	0001
2	00000 00100	0010
3	00000 01000	0011
4	00000 10000	0100
5	00001 00000	0101
6	00010 00000	0110
7	00100 00000	0111
8	01000 00000	1000
9	10000 00000	1001
10		
11		

下面均未选用

BCD码：用二进制代码对十进制数进行编码，

既有二进制码的形式(4位二进制码)，

又有十进制数的特点(每4位二进制码为1位十进制数)。

$$(185)_{10} = (0001_1000_0101)_{BCD}$$

$$(185)_{10} = (10111001)_{2}$$

Decimal Symbol	BCD Representation
10	0001 0000
11	0001 0001
212	0010 0001 0010
213	0010 0001 0011
5673	0101 0110 0111 0011
5684	0101 0110 1000 0100

BCD 加法

4		0100		3		0011		9		1001
+5		0101		+7		0111		+9		1001
9		1001		10		1010		18		10010
						+0110				+0110
						0001 0000				0001 1000

如果二进制的和 ≥ $(1010)_2$，就需要再加上$(0110)_2$.

Multi-Digit BCD Addition

BCD Carry	1	1		
	0010	1001	0101	295
	0110	0011	0101	+635
Binary Sum	1001	1101	1010	
Add 6		0110	0110	
BCD Sum	1001	0011	0000	930

BCD 减法

- 采用10的补码表示负数
- **符号位**：**0**～正数，**9**～负数

例：　　(+257)+(-160) =+97　　1000-160 = **840** (160的补)

	1
0 257	0000 0010 0101 0111
9 840	1001 1000 0100 0000
	1010 1010 1001 0111
	0110 0110
0 097	0000 0000 1001 0111

其他形式的BCD码

十进制数	8421码 有权码	2421码 有权码	余3码 偏权码	8 4 -2 -1 码 正负权值码
0	0000	0000	0011	0 0 0 0
1	0001	0001	0100	0 1 1 1
2	0010	0010	0101	0 1 1 0
3	0011	0011	0110	0 1 0 1
4	0100	0100	0111	0 1 0 0
5	0101	1011	1000	1 0 1 1
6	0110	1100	1001	1 0 1 0
7	0111	1101	1010	1 0 0 1
8	1000	1110	1011	1 0 0 0
9	1001	1111	1100	1 1 1 1

8421码 + 3

十进制也可以取反了，如2与7

自反码

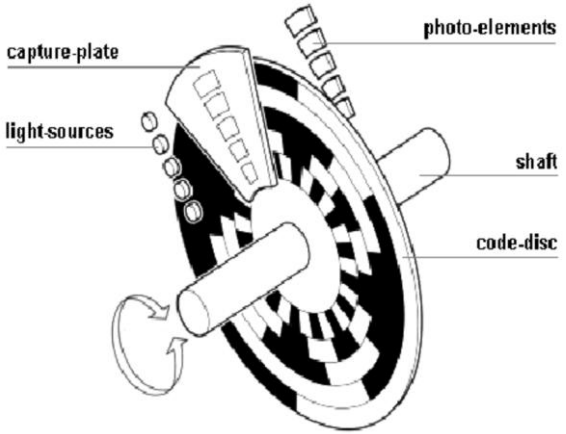
(ASCII码) 美国信息交换标准代码

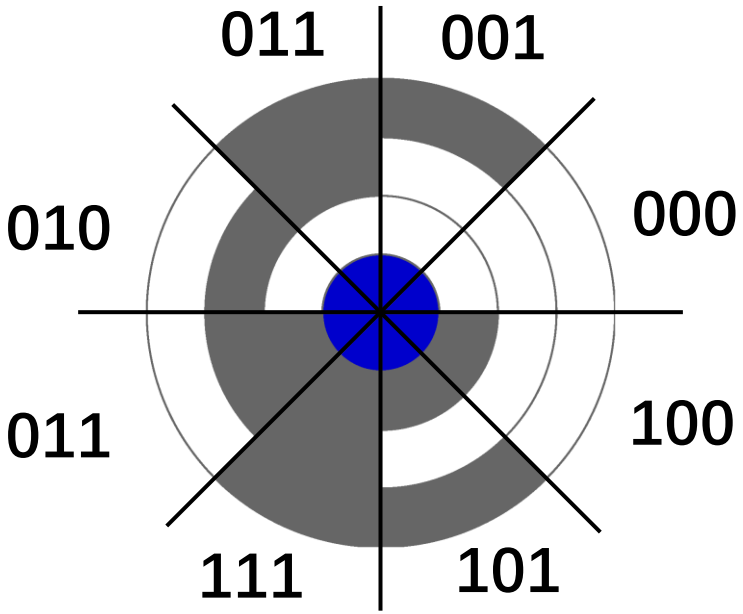
Dec	Hex	Name	Char	Ctrl-char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0	Null	NUL	CTRL-@	32	20	Space	64	40	@	96	60	`
1	1	Start of heading	SOH	CTRL-A	33	21	!	65	41	A	97	61	a
2	2	Start of text	STX	CTRL-B	34	22	"	66	42	B	98	62	b
3	3	End of text	ETX	CTRL-C	35	23	#	67	43	C	99	63	c
4	4	End of xmit	EOT	CTRL-D	36	24	$	68	44	D	100	64	d
5	5	Enquiry	ENQ	CTRL-E	37	25	%	69	45	E	101	65	e
6	6	Acknowledge	ACK	CTRL-F	38	26	&	70	46	F	102	66	f
7	7	Bell	BEL	CTRL-G	39	27	'	71	47	G	103	67	g
8	8	Backspace	BS	CTRL-H	40	28	(	72	48	H	104	68	h
9	9	Horizontal tab	HT	CTRL-I	41	29	)	73	49	I	105	69	i
10	0A	Line feed	LF	CTRL-J	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	VT	CTRL-K	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	FF	CTRL-L	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage feed	CR	CTRL-M	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	SO	CTRL-N	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	SI	CTRL-O	47	2F	/	79	4F	O	111	6F	o
16	10	Data line escape	DLE	CTRL-P	48	30	0	80	50	P	112	70	p
17	11	Device control 1	DC1	CTRL-Q	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	DC2	CTRL-R	50	32	2	82	52	R	114	72	r
19	13	Device control 3	DC3	CTRL-S	51	33	3	83	53	S	115	73	s
20	14	Device control 4	DC4	CTRL-T	52	34	4	84	54	T	116	74	t
21	15	Neg acknowledge	NAK	CTRL-U	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	SYN	CTRL-V	54	36	6	86	56	V	118	76	v
23	17	End of xmit block	ETB	CTRL-W	55	37	7	87	57	W	119	77	w
24	18	Cancel	CAN	CTRL-X	56	38	8	88	58	X	120	78	x
25	19	End of medium	EM	CTRL-Y	57	39	9	89	59	Y	121	79	y
26	1A	Substitute	SUB	CTRL-Z	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	ESC	CTRL-[	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	FS	CTRL-\	60	3C	<	92	5C	\	124	7C	|
29	1D	Group separator	GS	CTRL-]	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	RS	CTRL-^	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	US	CTRL-_	63	3F	?	95	5F	_	127	7F	DEL

- 最常用的一种字符编码
- 7位二进制数表达128个字符
- 最左侧增加1位奇偶校验位

格雷码 （可靠性编码）

角度传感器





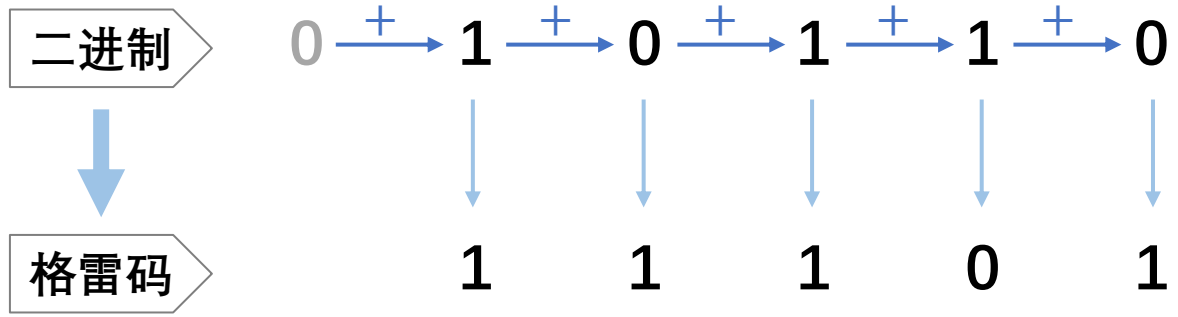
1940贝尔实验室Frank Gray提出，1953申请专利公开

- 任意相邻两数只有一位发生变化。
- 最高位的0和1只改变一次。

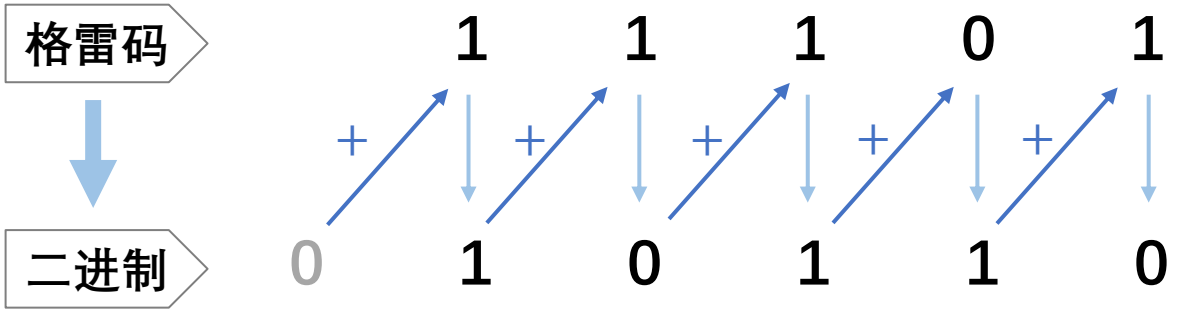
0	0	0	
0	0	1	
			对称轴
0	1	1	
0	1	0	
			对称轴
1	1	0	
1	1	1	
1	0	1	
1	0	0	

十进制数	8421码	格雷码
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

格雷码 Gray Code



（舍去进位）



变权码，每一位码没有固定的大小，

很难直接进行比较大小和算术运算。

奇偶校验码 Parity Check Code

校验码							
P	b_7	b_6	b_5	b_4	b_3	b_2	b_1
校验位	被 校 验 位						

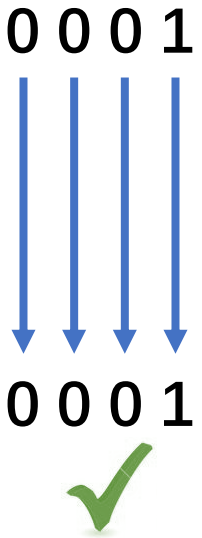
- 用1位***P***来表示奇偶性。根据需要设置为0或1，**使1的总数为偶数或奇数**。
- 只具有发现出现奇数个错误的能力，无错误定位、纠正错误的能力。

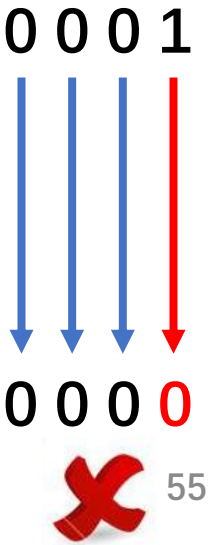
Even 偶校验，当b_i中1的个数是偶数时，$P=0$

当b_i中1的个数是奇数时，$P=1$

Odd 奇校验，当b_i中1的个数是奇数时，$P=0$

否则，$P=1$





Hamming 汉明码 只能检测和纠正1个错误

- An 8 bit data word is stored in memory with 4 parity bits as a 12 bit word:
  - 1 2 3 4 5 6 7 8 9 10 11 12
  - P_1 P_2 1 P_4 1 0 0 P_8 0 1 0 0

P_1 = XOR of bits $(3, 5, 7, 9, 11) = 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 0$

P_2 = XOR of bits $(3, 6, 7, 10, 11) = 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 0$

P_4 = XOR of bits $(5, 6, 7, 12) = 1 \oplus 0 \oplus 0 \oplus 0 = 1$

P_8 = XOR of bits $(9, 10, 11, 12) = 0 \oplus 1 \oplus 0 \oplus 0 = 1$

C_1 = XOR of bits $(1, 3, 5, 7, 9, 11)$

C_2 = XOR of bits $(2, 3, 6, 7, 10, 11)$

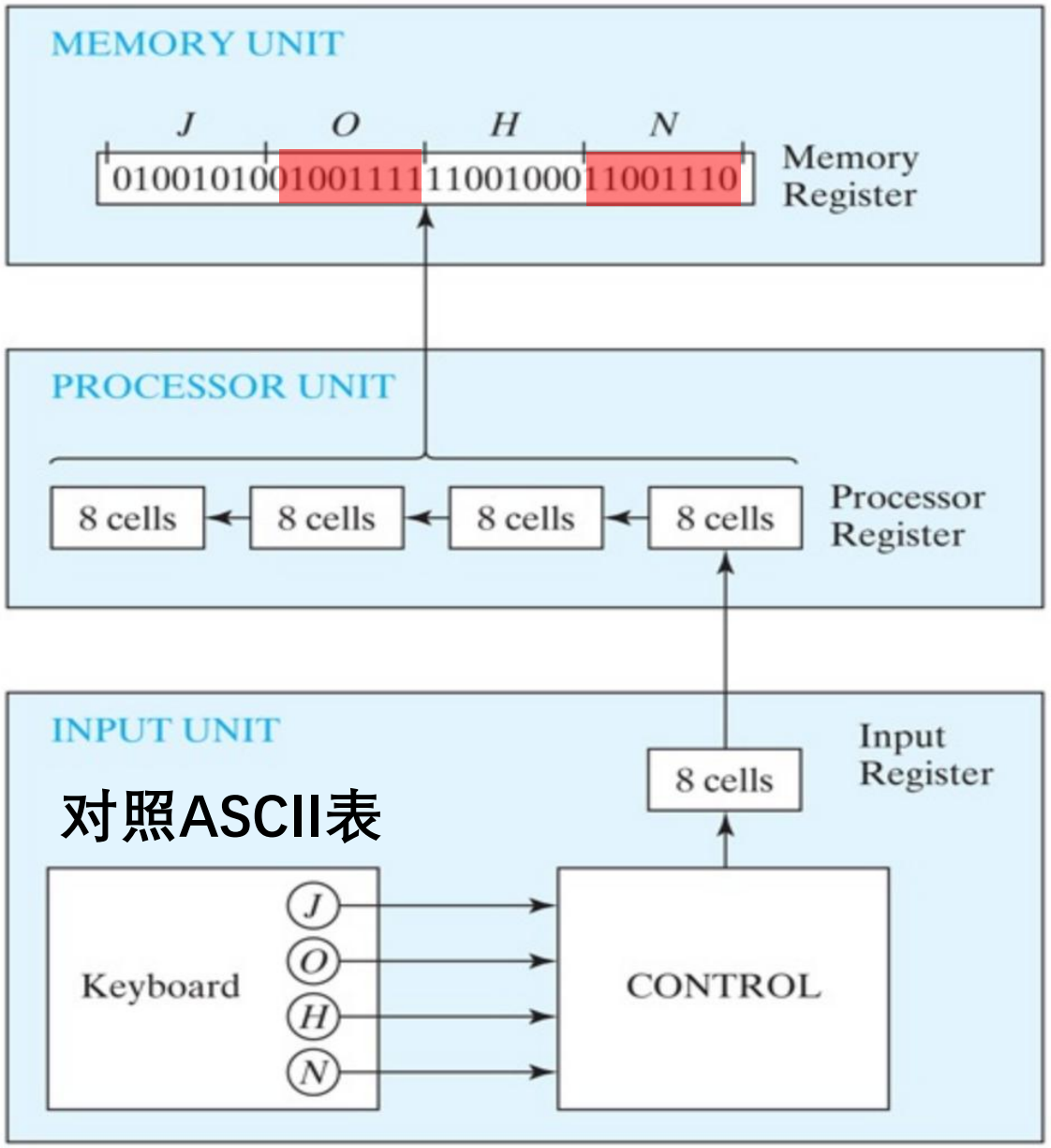
C_4 = XOR of bits $(4, 5, 6, 7, 12)$

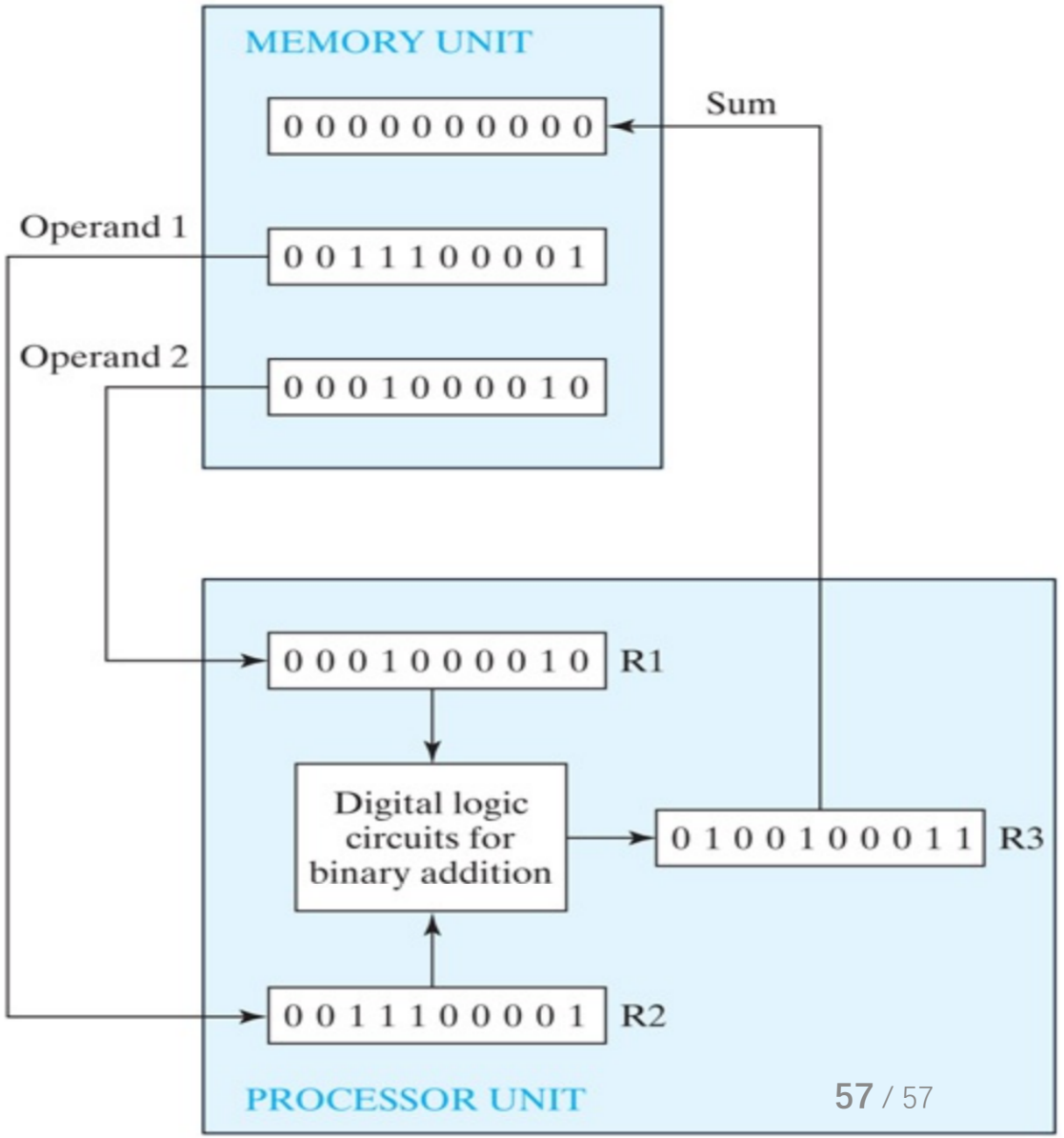
C_8 = XOR of bits $(8, 9, 10, 11, 12)$

- The result of the parity bits as:
  - 1 2 3 4 5 6 7 8 9 10 11 12
  - 0 0 1 1 1 0 0 1 0 1 0 0

Bit position:	1	2	3	4	5	6	7	8	9	10	11	12		
	0	0	1	1	1	0	0	1	0	1	0	0	No error	$C_8C_4C_2C_1 = \mathbf{0000}$ 没错误
	X	0	1	1	1	0	0	1	0	1	0	0	Error in bit 1	$C_8C_4C_2C_1 = \mathbf{0001}$
	0	0	1	1	X	0	0	1	0	1	0	0	Error in bit 5	$C_8C_4C_2C_1 = \mathbf{0101}$

二进制应用实例


MEMORY UNIT
J
O
H
N
01001010 01001111 11001000 11001110
Memory Register
PROCESSOR UNIT
8 cells
8 cells
8 cells
8 cells
Processor Register
INPUT UNIT
Input Register
8 cells
对照ASCII表
Keyboard
J
O
H
N
CONTROL


MEMORY UNIT
0 0 0 0 0 0 0 0 0 0 0
Sum
Operand 1
0 0 1 1 1 0 0 0 0 1
Operand 2
0 0 0 1 0 0 0 0 1 0
0 0 0 1 0 0 0 0 1 0
R1
Digital logic circuits for binary addition
0 1 0 0 1 0 0 0 1 1
R3
0 0 1 1 1 0 0 0 0 1
R2
PROCESSOR UNIT